

MATLAB LAB 19 | مختبر MATLAB 19

AI-Based Intrusion Detection

كشف الهجمات السيبرانية باستخدام الذكاء الاصطناعي

Synthetic PMU data, false-data attacks, SVM classification, and robust evaluation

بيانات PMU اصطناعية وهجمات بيانات كاذبة وتصنيف SVM وتقييم متين

Prepared by | إعداد: Dr.-Ing. Aouss Gabash | د.م. أوس غباش

Objectives

- Generate normal and attacked power-system measurements.
- Construct meaningful electrical and residual features.
- Split data into training and test sets without leakage.
- Train an SVM intrusion detector.
- Compute confusion matrix, precision, recall, F1, and false-alarm rate.
- Study noise, class imbalance, threshold selection, and unseen attacks.
- Compare data-driven and physics-informed detection.

الأهداف

- توليد قياسات طبيعية ومهاجمة لنظام قدرة.
- بناء ميزات كهربائية وبواقي مفيدة.
- تقسيم البيانات إلى تدريب واختبار دون تسرب.
- تدريب كاشف اختراق باستخدام SVM.
- حساب مصفوفة الالتباس و Precision و Recall و F1 ومعدل الإنذار الكاذب.
- دراسة الضوضاء وعدم توازن الفئات واختيار العتبة والهجمات غير المرئية.
- مقارنة الكشف المعتمد على البيانات بالكشف المدعوم بالفيزياء.

1. Laboratory Scenario

We simulate PMU-like measurements for voltage magnitude, phase angle, frequency deviation, and active power. Attack samples contain coordinated measurement shifts and occasional replay-like behavior.

$$z_{attack} = z + a + v$$

The detector classifies each observation as normal or attacked.

1. سيناريو المختبر

نحاكي قياسات شبيهة بوحدات PMU تشمل الجهد والزاوية وانحراف التردد والقدرة الفعالة، ثم نضيف هجمات منسقة وانحرافات وإعادة إرسال.

2. Complete MATLAB Program

```

clear; clc; close all;
rng(19);

%% Data size
Nnormal = 1800;
Nattack = 600;

%% Normal PMU-like measurements
Vn = 1.00 + 0.008*randn(Nnormal,1);
An = 0.00 + 0.020*randn(Nnormal,1); % angle [rad]
Fn = 0.00 + 0.015*randn(Nnormal,1); % frequency deviation [Hz]
Pn = 1.10 + 0.12*randn(Nnormal,1); % active power [p.u.]

%% Attack measurements
Va = 1.00 + 0.008*randn(Nattack,1);
Aa = 0.00 + 0.020*randn(Nattack,1);
Fa = 0.00 + 0.015*randn(Nattack,1);
Pa = 1.10 + 0.12*randn(Nattack,1);

% Coordinated false-data shifts
Va = Va + 0.035 + 0.010*randn(Nattack,1);
Aa = Aa + 0.090 + 0.020*randn(Nattack,1);
Fa = Fa + 0.060 + 0.020*randn(Nattack,1);
Pa = Pa + 0.28 + 0.08*randn(Nattack,1);

%% Physics-inspired residual feature
% Simple approximate relation:  $P = c1 \cdot \text{angle} + c2 \cdot (1-V)$ 
c1 = 8.0;
c2 = 3.0;
Rn = Pn - (1.10 + c1*An + c2*(1-Vn));
Ra = Pa - (1.10 + c1*Aa + c2*(1-Va));

%% Feature matrix and labels
Xnormal = [Vn An Fn Pn Rn];
Xattack = [Va Aa Fa Pa Ra];
X = [Xnormal; Xattack];
Y = [zeros(Nnormal,1); ones(Nattack,1)];

%% Random train/test split
cv = cvpartition(Y,'HoldOut',0.30);
Xtrain = X(training(cv),:);
Ytrain = Y(training(cv));
Xtest = X(test(cv),:);
Ytest = Y(test(cv));

```

```

%% Standardization using training statistics only
mu = mean(Xtrain,1);
sigma = std(Xtrain,[],1);
sigma(sigma==0) = 1;
XtrainZ = (Xtrain-mu)./sigma;
XtestZ = (Xtest-mu)./sigma;

%% Train SVM classifier
mdl = fitcsvm(XtrainZ,Ytrain,...
    'KernelFunction','rbf',...
    'KernelScale','auto',...
    'Standardize',false,...
    'ClassNames',[0 1]);
mdl = fitPosterior(mdl,XtrainZ,Ytrain);

%% Test predictions
[pred,score] = predict(mdl,XtestZ);

%% Confusion matrix and metrics
cm = confusionmat(Ytest,pred,'Order',[0 1]);
TN = cm(1,1); FP = cm(1,2);
FN = cm(2,1); TP = cm(2,2);

accuracy = (TP+TN)/sum(cm,'all');
precision = TP/max(TP+FP,1);
recall = TP/max(TP+FN,1);
F1 = 2*precision*recall/max(precision+recall,eps);
falseAlarmRate = FP/max(FP+TN,1);

fprintf('Accuracy = %.3f\n',accuracy);
fprintf('Precision = %.3f\n',precision);
fprintf('Recall = %.3f\n',recall);
fprintf('F1 score = %.3f\n',F1);
fprintf('False alarm rate = %.3f\n',falseAlarmRate);
disp('Confusion matrix [TN FP; FN TP]:');
disp(cm);

%% Confusion chart
figure;
confusionchart(Ytest,pred,...
    'RowSummary','row-normalized',...
    'ColumnSummary','column-normalized');
title('AI Intrusion Detector');

%% ROC curve

```

```
[fpRate, tpRate, ~, AUC] = perfcurve(Ytest, score(:, 2), 1);
figure;
plot(fpRate, tpRate, 'LineWidth', 1.8); hold on;
plot([0 1], [0 1], '--');
grid on;
xlabel('False Positive Rate');
ylabel('True Positive Rate');
title(sprintf('ROC Curve, AUC = %.3f', AUC));

%% Feature visualization
figure;
gscatter(Xtest(:, 5), Xtest(:, 3), Ytest);
grid on;
xlabel('Physics residual');
ylabel('Frequency deviation');
title('Normal and Attack Samples');
```

2. البرنامج الكامل

ينشئ البرنامج بيانات طبيعية ومهاجمة، ويضيف ميزة بواقى مستوحاة من الفيزياء، ويقسم البيانات ويطبّعها اعتمادًا على التدريب فقط، ثم يدرب SVM ويحسب مؤشرات التقييم.

3. Why the Residual Feature Helps

$$r_P = P - \hat{P}(V, \theta)$$

Raw measurements may overlap, but an attack can violate the expected physical relationship among voltage, angle, and power. The residual exposes this inconsistency.

Physics-informed features often improve generalization when attack data are limited.

3. أهمية ميزة البواقى

قد تتداخل القياسات الخام بين الحالة الطبيعية والهجوم، لكن البواقى تكشف عدم الاتساق بين الجهد والزاوية والقدرة وفق النموذج الفيزيائي.

4. Experiment A: Remove the Physics Feature

```
XtrainNoR = XtrainZ(:,1:4);
XtestNoR  = XtestZ(:,1:4);

mdlNoR = fitcsvm(XtrainNoR,Ytrain,...
    'KernelFunction','rbf','KernelScale','auto');
predNoR = predict(mdlNoR,XtestNoR);
cmNoR = confusionmat(Ytest,predNoR,'Order',[0 1]);
disp(cmNoR);
```

Compare recall and false-alarm rate with and without the residual feature.

4. التجربة A: حذف الميزة الفيزيائية

أعد التدريب دون البواقي وقارن الاستدعاء والإنذارات الكاذبة لتحديد قيمة المعلومات الفيزيائية.

5. Experiment B: Class Imbalance

Reduce the attack class to 5% of all samples. Accuracy may remain high even when the detector misses many attacks.

```
attackKeep = randperm(Nattack,round(0.05*Nnormal));
Ximbalanced = [Xnormal; Xattack(attackKeep,:)];
Yimbalanced = [zeros(Nnormal,1); ones(numel(attackKeep),1)];
```

Always report recall, precision, F1, false-alarm rate, and the confusion matrix.

5. التجربة B: عدم توازن الفئات

قلل الهجمات إلى نسبة صغيرة ولاحظ كيف يمكن أن تبدو الدقة العامة مرتفعة رغم ضعف اكتشاف الهجمات.

6. Experiment C: Noise Robustness

```
noiseLevels = [0 0.25 0.50 1.00];
results = zeros(numel(noiseLevels),2);

for q = 1:numel(noiseLevels)
    Xnoisy = XtestZ + noiseLevels(q)*randn(size(XtestZ));
    predQ = predict mdl,Xnoisy);
    cmQ = confusionmat(Ytest,predQ,'Order',[0 1]);
    results(q,1) = (cmQ(1,1)+cmQ(2,2))/sum(cmQ,'all');
    results(q,2) = cmQ(2,2)/max(sum(cmQ(2,:)),1);
end

disp(table(noiseLevels',results(:,1),results(:,2),...
    'VariableNames',{'Noise','Accuracy','Recall'}));
```

6. التجربة C: المتانة للضوضاء

أضف مستويات مختلفة من الضوضاء إلى بيانات الاختبار وراقب تراجع الدقة والاستدعاء.

7. Experiment D: Detection Threshold

```

probAttack = score(:,2);
thresholds = 0.10:0.05:0.90;
metricTable = zeros(numel(thresholds),3);

for q = 1:numel(thresholds)
    predT = probAttack >= thresholds(q);
    cmT = confusionmat(Ytest,predT,'Order',[0 1]);
    TN = cmT(1,1); FP = cmT(1,2);
    FN = cmT(2,1); TP = cmT(2,2);
    metricTable(q,1) = TP/max(TP+FN,1); % recall
    metricTable(q,2) = FP/max(FP+TN,1); % false alarm
    metricTable(q,3) = 2*TP/max(2*TP+FP+FN,1); % F1
end

figure;
plot(thresholds,metricTable,'LineWidth',1.5);
grid on;
xlabel('Decision threshold');
ylabel('Metric value');
legend('Recall','False alarm rate','F1');
title('Threshold Selection');

```

7. التجربة D: عتبة القرار

غير عتبة التصنيف ولاحظ المفاضلة بين اكتشاف المزيد من الهجمات وتقليل الإنذارات الكاذبة.

8. Experiment E: Unseen Attack Type

```

Nunseen = 300;
Vu = 1.00 + 0.008*randn(Nunseen,1);
Au = 0.00 + 0.020*randn(Nunseen,1);
Fu = 0.00 + 0.015*randn(Nunseen,1);
Pu = 1.10 + 0.12*randn(Nunseen,1);

% Replay-like attack: frequency looks normal, angle-power relation is
altered
Au = Au - 0.07;
Pu = Pu + 0.22;
Ru = Pu - (1.10 + c1*Au + c2*(1-Vu));
Xu = [Vu Au Fu Pu Ru];
XuZ = (Xu-mu)./sigma;
predUnseen = predict mdl,XuZ;

fprintf('Detection rate for unseen attack = %.3f\n',mean(predUnseen));

```

This test measures generalization beyond the exact attack distribution used during training.

8. التجربة E: نوع هجوم غير مرئي

اختبر النموذج على هجوم لم يظهر في التدريب، لأن الأداء على الهجمات المعروفة لا يضمن التعميم.

9. Experiment F: Adversarial Evasion

```
XattackTest = XtestZ(Ytest==1,:);
Xevade = XattackTest;

% Simple illustrative perturbation toward normal feature mean
normalMean = mean(XtrainZ(Ytrain==0,:),1);
epsilon = 0.15;
Xevade = Xevade + epsilon*sign(normalMean-Xevade);

predBefore = predict mdl,XattackTest;
predAfter  = predict mdl,Xevade;

fprintf('Attack detection before perturbation = %.
3f\n',mean(predBefore));
fprintf('Attack detection after perturbation  = %.
3f\n',mean(predAfter));
```

This is an educational evasion simulation, not a full gradient-based adversarial attack.

9. التجربة F: المراوغة الخصومية

حرّك ميزات الهجوم قليلاً باتجاه متوسط البيانات الطبيعية وراقب تغير نسبة الكشف. التجربة تعليمية لتوضيح حساسية المصنف.

10. Compare Several Models

```
% Decision tree
mdlTree = fitctree(XtrainZ,Ytrain);
predTree = predict(mdlTree,XtestZ);

% Ensemble of trees
mdlEns = fitcensemble(XtrainZ,Ytrain,'Method','Bag');
predEns = predict(mdlEns,XtestZ);

% Linear discriminant
mdlLDA = fitcdiscr(XtrainZ,Ytrain);
predLDA = predict(mdlLDA,XtestZ);

models = {'SVM';'Tree';'Ensemble';'LDA'};
preds = {pred;predTree;predEns;predLDA};
Flall = zeros(4,1);

for m = 1:4
    cmM = confusionmat(Ytest,preds{m},'Order',[0 1]);
    TN = cmM(1,1); FP = cmM(1,2);
    FN = cmM(2,1); TP = cmM(2,2);
    Flall(m) = 2*TP/max(2*TP+FP+FN,1);
end

disp(table(models,Flall));
```

10. مقارنة نماذج متعددة

قارن SVM بشجرة قرار وتجميع أشجار ومصنف خطي باستخدام F1 وليس الدقة العامة فقط.

11. Tasks

1. Change attack magnitude and class ratio.
2. Add measurement spikes, drift, replay, and packet-loss features.
3. Use cross-validation to tune SVM hyperparameters.
4. Compare raw, statistical, temporal, and physics-informed features.
5. Study detection delay using sliding windows.
6. Build a one-class anomaly detector trained only on normal data.
7. Test robustness under an unseen topology or operating point.

8. Report accuracy, precision, recall, F1, AUC, and false-alarm rate.

11. المهام

1. غير شدة الهجوم ونسبة الفئات.
2. أضف النبضات والانجراف وإعادة الإرسال وفقدان الحزم.
3. استخدم التحقق المتقاطع لضبط SVM.
4. قارن الميزات الخام والإحصائية والزمنية والفيزيائية.
5. ادرس تأخير الكشف باستخدام النوافذ المنزلقة.
6. ابن كاشف شذوذ يتدرب على البيانات الطبيعية فقط.
7. اختبر المتانة عند نقطة تشغيل أو بنية شبكة جديدة.
8. اعرض جميع مؤشرات التقييم الأساسية.

12. Mini Project

Develop a PMU cyberattack detector for a small multi-bus system. Generate time-series measurements under normal operation, load changes, line outages, and several false-data attacks. Combine an electrical residual detector with an AI classifier, evaluate unseen attacks, and design a safe operator alarm policy.

12. المشروع المصغر

طوّر كاشف هجمات لقياسات PMU في شبكة متعددة العقد، مع تشغيل طبيعي وتغيير حمل وفصل خط وعدة هجمات. ادمج كاشف بواقى كهربائياً مع مصنف ذكاء اصطناعي واختبر هجمات غير مرئية وصمم سياسة إنذار أمانة للمشغل.

Publication Information

معلومات النشر والمرجع

Document	Laboratory 19 · AI-Based Intrusion Detection
Course	AI Applications in Electrical Power Systems
Author	Dr.-Ing. Aouss Gabash
Version	1.0
Publication year	2026
Official website	https://aoussgabash.com

Recommended citation:
Gabash, A. (2026). AI-Based Intrusion Detection. AI Applications in Electrical Power Systems, Laboratory 19, Version 1.0. <https://aoussgabash.com>

المرجع المقترح:
أوس غباش، 2026، مخبر 19، تطبيقات الذكاء الاصطناعي في أنظمة الطاقة الكهربائية، الإصدار 1.0، <https://aoussgabash.com>

© 2026 Dr.-Ing. Aouss Gabash. Educational use with attribution to the author and official website.